Exercise 1 – Wait for the First Child

The parent created two children. Child 1 slept for 3 seconds and Child 2 slept for 1 second. The program was run three times. Child 2 finished first in all three recorded runs because it slept for less time. The parent's single wait() call returned Child 2's PID and exit status 2 each time. The parent then exited without waiting for Child 1; Child 1 finished afterward. This demonstrates that wait() collects one terminated child, not necessarily every child. The operating system still does not guarantee a general process scheduling order.

Exercise 2 – Parent, Child, and Grandchild

The output showed a three-level hierarchy: the grandchild's PPID matched the child's PID, and the child's PPID matched the original parent's PID. The grandchild finished first. The child collected the grandchild with waitpid(), then exited. The original parent collected the child and printed its final message only after both descendants had finished.

Exercise 3 – Wait for All Children

The parent created three children with sleep durations of 3, 2, and 1 seconds. In the recorded run, the start messages appeared in the order Child 2, Child 3, Child 1, while the children finished in the order Child 3, Child 2, Child 1. The parent called wait() three times and reported each collected PID and exit status (3, 2, and 1). The parent printed its final message after all three children had been collected.

Exercise 4 – Wait for a Specific Child

The parent first used waitpid() for the second-created child (children[1]). In the recorded run, Child 3 finished before Child 2, but the parent collected Child 2 first because it was waiting for that specific PID. Child 3 remained terminated but unreaped until a later wait() collected it. The second wait() collected Child 1. This shows that collection order can differ from termination order.

Exercise 5 – Analysis

1. Does fork() guarantee the order in which children run? Explain.

No. fork() creates a child process, but it does not guarantee which process runs first or the order in which the parent and children execute. The operating system scheduler determines when each process runs. Therefore, output order can vary between executions. Different sleep durations influence when children are expected to finish, but they do not create a general scheduling guarantee.

2. How does waitpid(pid, ...) differ from wait()?

wait() waits for any one child process to terminate and returns the PID of the child it collected. waitpid(pid, ...) waits for a particular child identified by PID (when called with options set to 0, it blocks until that child terminates). Thus, wait() accepts whichever child finishes, while waitpid() lets the parent select which child to wait for.

3. Can a child terminate before its parent calls wait()? What happens then?

Yes. If a child terminates before its parent collects it, the child normally remains as a zombie process. The operating system retains its termination status so the parent can retrieve it later with wait() or waitpid(). Once collected, the child is reaped and its process-table entry can be released.

4. What happens if a parent terminates while a child is still running?

The child becomes an orphan process. It can continue running, and the operating system reparents it to an appropriate system process or configured subreaper. The original parent can no longer wait for that child. Exercise 1 demonstrates this situation when the parent waits for only one of its two children and exits while the other child is still running.

5. Why does the intermediate child in Exercise 2 wait for the grandchild?

The grandchild is a direct child of the intermediate child, not of the original parent. A process can wait for its own child, but the original parent cannot directly waitpid() for the grandchild. Therefore, the intermediate child must wait for and collect the grandchild. The original parent then waits for the intermediate child, ensuring that the original parent exits only after both descendants have finished.

4. Conclusion

This lab demonstrated process creation with fork(), process identification with getpid() and getppid(), and child-process collection with wait() and waitpid(). The exercises showed that process execution and termination order are not guaranteed, that wait() collects any child while waitpid() can target a selected child, and that each parent process is responsible for collecting its own children.